AFRICAN LEADERSHIP UNIVERSITY

# Protecting the ALU Logo on the Ethereum Blockchain

*Blockchain and Applications — Assignment 1*
*Question 1: Cryptocurrency Exchange Exploration*
*Question 2: Smart Contracts for Logo Registration and Tokenisation*

*October 2026*




*Figure 1. Official ALU wordmark registered on-chain (source: naissance.alueducation.com).*

## 1. Abstract

This project explores how a regulated cryptocurrency exchange works and then uses that same public-ledger idea to protect a real digital asset: the official ALU logo. Two Solidity contracts were written, tested, and deployed on a local Hardhat Ethereum network. `ALUAssetRegistry` records the logo as an ERC-721 NFT together with its SHA-256 file fingerprint, so a fake or edited copy can be detected. `ALULogoToken` then issues 1,000,000 ERC-20 ALUT shares so ownership of that unique asset can be split among ALU stakeholders without changing the logo file itself.

## 2. Introduction

ALU's logo is how students, employers, and the public recognise the university. Once the file is on the internet it can be copied, recoloured, or attached to a fake certificate. A normal database can store 'ALU owns this file', but the operator of that database can edit the row later, and outsiders have to trust the operator. A copyright registration proves a legal claim, but it does not let a stranger check whether the PNG on their laptop is the same bytes ALU originally published.

Putting the logo on Ethereum solves a different problem. The SHA-256 hash of the exact file is written into a smart contract. After that, nobody — not even the contract author — can quietly swap the hash, because changing contract storage would require a new transaction that every node would reject unless the contract itself exposed a rewrite function (this one does not). Anyone can hash a suspect file and compare it with the on-chain value. The NFT also records who registered the logo and when. Tokenisation then lets ALU issue shares of that unique asset without handing the original file to every stakeholder.

## 3. Question 1: Exploring Coinbase

*Exchange exploration report (626 words).*

This section walks through Coinbase for a classmate who has never used an exchange. No real money was spent. The six steps follow Coinbase's public product and official help documentation.

**Step 1 — Create an account.** You start with an email, a password, your country, and a confirmation that you are 18 or older. Coinbase then runs Know Your Customer checks: legal name, date of birth, home address, a government ID (passport, national ID, or driving licence), and a selfie matched to that ID. Some regions also ask about employment and the source of funds. Exchanges require this because they are regulated like financial firms. In the United States the Bank Secrecy Act, the USA PATRIOT Act, and FinCEN AML rules force them to identify customers and screen sanctions lists. Other countries apply similar KYC/AML law under FATF guidance, so stolen identities and criminal funds are harder to move through the platform.

**Step 2 — Explore the platform.** The dashboard shows portfolio value and buy/sell shortcuts. The wallet lists each crypto and cash balance. Trading has a simple Buy screen and an Advanced Trade order book. Bitcoin (BTC) is digital cash with a 21 million cap; people use it as a store of value, and unlike Ethereum it is not a general-purpose computer. Ethereum (ETH) runs smart contracts and NFTs — including this assignment's contracts — and ETH is spent as gas. Solana (SOL) is also programmable, but it is built for speed: cheap, sub-second transfers rather than Bitcoin's conservatism or Ethereum's default token standards.

**Step 3 — Add a payment method.** Settings → Payment methods lets you link a bank account or debit card. A bank account is linked through a service such as Plaid, or by entering account and routing numbers and confirming micro-deposits. A debit card needs the number, expiry, CVC, and billing address, and the name must match the verified account. Bank transfer is cheaper (typically no funding fee) but slower to clear. A debit card is instant but adds about 3.99 percent. On $100 that is roughly four extra dollars, so the bank transfer is better value if you can wait.

**Step 4 — Simulate a $100 purchase.** I would buy Ethereum, the chain used in Question 2. A market order fills immediately at the current price. A limit order waits for a price you set. I would choose a market order: $100 is too small to wait for a slightly better fill, and I want a finished trade in history. On the simple Buy screen a bank-funded $100 order typically takes about $2.99 plus a ~0.5 percent spread. At an ETH price of about $2,363 on 28 October 2026, roughly $96.51 would buy about 0.0408 ETH after fees.

**Step 5 — Review the transaction record.** History shows the asset, buy or sell, amount, price, fees, payment method, status, and time, plus an internal Coinbase reference. When crypto later moves onto a blockchain, the record also shows a transaction hash: a unique hex fingerprint of that on-chain transfer. It matters because anyone can paste it into a block explorer and independently check sender, receiver, amount, and block. Buys that stay inside Coinbase are internal ledger entries and may not have a chain hash until you withdraw.

**Step 6 — Security features.** Two-factor authentication asks for a code from an authenticator app or hardware key after your password, so a stolen password is not enough. Coinbase prefers that over SMS, which can be SIM-swapped. Withdrawal allowlists and limits are the second control: funds can only leave to pre-approved addresses, and amounts are capped until more verification is done. Even if an attacker logs in, they cannot instantly drain the account to a brand-new wallet.

Note on screenshots for Question 1. Completing Coinbase identity verification requires a real government ID and a live selfie. Those personal documents were not submitted from this development environment. The six steps above follow Coinbase's published product and help centre. If the course requires captures from a verified personal account, add those images from your own login before uploading the PDF.

## 4. Blockchain Setup

A local blockchain is a complete Ethereum network that runs only on your computer. Hardhat creates one in memory for tests, and `npx hardhat node` starts a longer-lived JSON-RPC node at `http://127.0.0.1:8545`. Developers use a local chain instead of Ethereum mainnet because every

mainnet transaction costs real ETH, is public forever, and cannot be undone. On Hardhat you get 20 pre-funded test wallets, blocks mine in milliseconds, and you can reset state as many times as you like. Nothing here touches the real Ethereum network.

A wallet address is a 20-byte identifier, usually shown as a 42-character hexadecimal string starting with `0x`. It is derived from a public key, which is derived from a private key. On the blockchain it represents an account: the place that holds ETH, owns NFTs, and is recorded as `msg.sender` when a contract is called. Account #0 below, `0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266`, is the well-known Hardhat deployer. Each account starts with 10,000 test ETH. Those private keys are public documentation keys — they must never receive real funds.

What each part of the setup is doing. Node.js is the runtime that Hardhat uses. Hardhat is the development environment: it compiles Solidity, runs tests, and exposes the local chain. OpenZeppelin provides audited ERC-721, ERC-20, and Ownable implementations so the assignment does not re-implement token standards from scratch. `hardhat.config.js` pins Solidity 0.8.24 and the Cancun EVM. The in-memory `hardhat` network is what `npx hardhat test` uses. The `localhost` network is what the deploy script uses once the node is running.

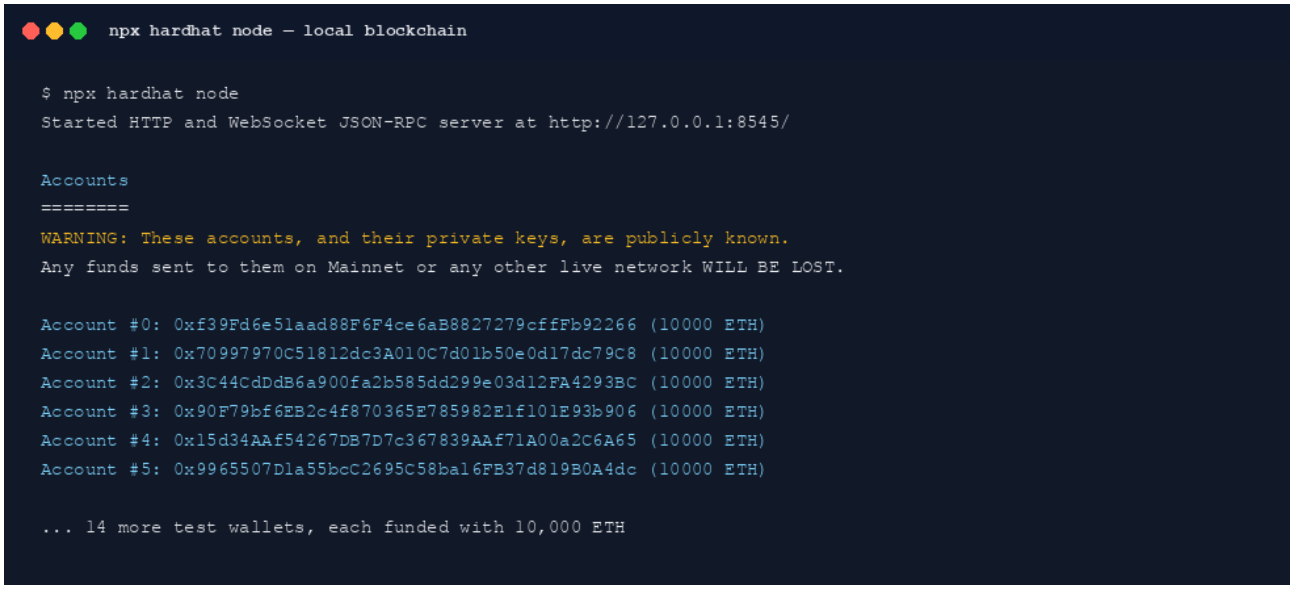



*Figure 2. Local Hardhat node at http://127.0.0.1:8545 with generated test wallet addresses.*

## 5. Smart Contract Explanation

A smart contract is a program stored on a blockchain. Once deployed, every honest node executes the same bytecode against the same state. That is different from a normal programme on a server. A server program can be stopped, patched, or made to return different answers to different users. A deployed contract has a public address, public code, and public storage. You cannot quietly log in and rewrite a row. If the contract has no function that changes a value, that value stays. Users do not trust ALU's web host; they trust the network's consensus rules.

An NFT (non-fungible token) is a token where each unit is unique. ERC-721 is the Ethereum standard for that idea: every token has an ID, a single owner, and optional metadata. The ALU logo is a single official file, not a pile of identical coins, so ERC-721 is the right standard. Two logos must not be interchangeable. ERC-20 would be the wrong choice for the file itself because any one ERC-20 unit is equal to any other.

### 5.1 Generating the ALU logo hash

Before registration the actual PNG was hashed with SHA-256 in Node.js (`crypto.createHash('sha256')`) and confirmed with `shasum -a 256`. SHA-256 is a cryptographic hash function: it takes any file and produces a fixed 32-byte digest. It is deterministic (the same file always yields the same digest) and collision-resistant (it is infeasible to find a different file with the same digest). Changing even one pixel flips many bits in the input, and the avalanche effect of SHA-256 produces a completely unrelated output. That is why a cropped, recoloured, or lightly edited fake logo cannot pass `verifyLogoIntegrity()`. The 32-byte digest fits naturally into Solidity's `bytes32` type.

```
SHA-256 = ceddec61dafedd84ad1a1a0d43e26de0a84ad71493a4c21b3405adbc6ec191db
```

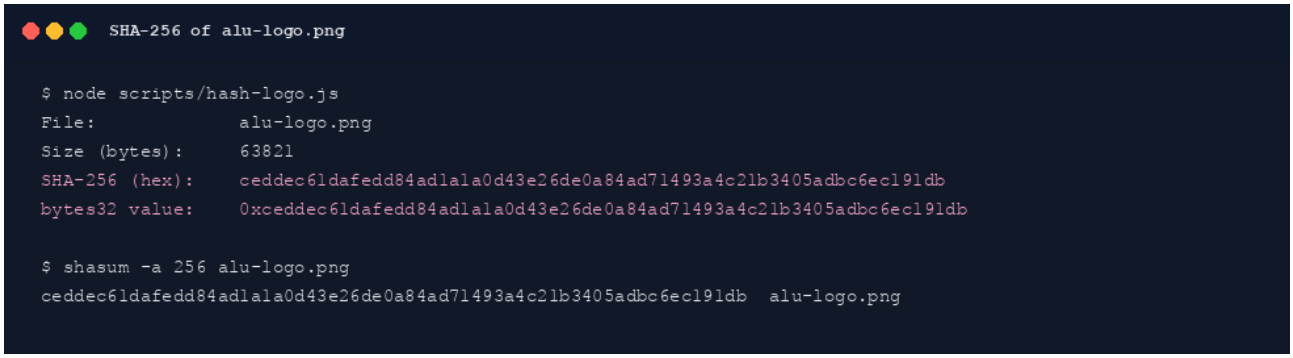



*Figure 3. SHA-256 fingerprint of alu-logo.png, generated from the real file, not a placeholder.*

### 5.2 ALUAssetRegistry (ERC-721)

The contract extends OpenZeppelin's ERC-721. `AssetMetadata` stores the asset name, file type, SHA-256 content hash, registrant address, and Unix timestamp. Mapping `assets` looks up that struct by token ID. Mapping `registeredHashes` records whether a hash has already been used.

**registerAsset(name, fileType, contentHash)** is the write function. It rejects an empty name, file type, or hash, then reverts with `"Asset already registered: duplicate content hash"` if the file was seen before. Otherwise it increments a counter from 1, mints an NFT to the caller with `_safeMint()`, writes the metadata, marks the hash as used, emits `AssetRegistered`, and returns the new token ID. The official logo is registered as token ID 1 during deployment.

**verifyLogoIntegrity(tokenId, contentHash)** is a `view` function, so a direct call from a wallet or dApp costs no gas. It loads the stored hash for that token and compares it with the caller's value. A match returns `(true, "Logo is authentic.")`. A mismatch returns `(false, "Warning: logo does not match.")`. That is the public authenticity check ALU wanted.

**getAsset(tokenId)** is also a view function. It returns the full `AssetMetadata` struct so anyone can read the name, file type, hash, registrant, and timestamp at any time.

Once stored, this data cannot be changed because the contract exposes no setter that overwrites `assets[tokenId]` or clears `registeredHashes`. Ethereum storage is append-only from the outside: the only way to mutate it is to execute a function the contract author wrote. With no such function, the registration is permanent for as long as the chain continues.

### 5.3 Deployment script

`scripts/deploy.js` hashes `alu-logo.png`, deploys `ALUAssetRegistry`, immediately calls `registerAsset("ALU Official Logo", "image/png", contentHash)`, then deploys `ALULogoToken` with the deployer as logo owner so the full ALUT supply is minted in the same run.

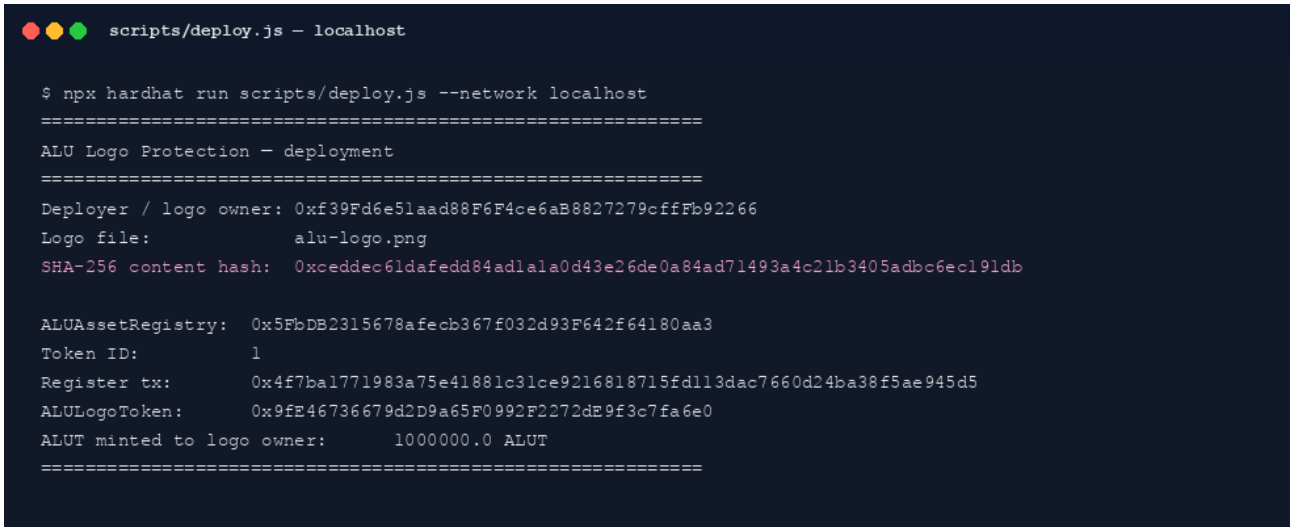



*Figure 4. Deployment on the local node: logo registered as token ID 1 and 1,000,000 ALUT minted.*

## 6. Tokenisation

Tokenisation means taking a unique real-world or digital asset and representing rights in that asset as digital tokens that can be held, transferred, or accounted for on a blockchain. For the ALU logo, the unique thing remains one NFT. What gets split is ownership. Instead of one office 'owning the brand file', many people can hold a measured share of it.

Who at ALU might receive ALUT, and why? The communications or brand office could keep a controlling treasury so the university remains the primary steward of the mark. Faculty who create official courseware that uses the logo might receive a small allocation as recognition that they are trusted users of the brand. Student government or campus ambassadors might hold a symbolic share tied to representing ALU in public. Administration (the registrar, legal, and partnerships teams) might hold shares because they approve how the logo appears on transcripts and partner materials. None of those people need a copy of a private PNG sitting in email. They hold tokens that the registry already ties to the authentic file.

ERC-721 and ERC-20 solve opposite problems. ERC-721 says 'this token is not interchangeable'. That is correct for one official logo file. ERC-20 says 'every unit is the same as every other unit'. That is correct for shares. One faculty member's 1 ALUT must equal another person's 1 ALUT, otherwise percentage ownership does not make sense. Using ERC-721 for the shares would make every share a unique object and destroy the idea of a simple percentage. Using ERC-20 for the logo file would imply that any one 'logo token' is as official as any other, which would defeat authenticity.

**ALULogoToken** is named "ALU Logo Token" with symbol ALUT. The constructor mints 1,000,000 tokens (18 decimals, so 1,000,000 × $10^{18}$ base units) to the logo owner and sets that address as `Ownable` owner. No further minting function exists, so the supply is fixed.

**distributeShares(recipient, amount)** can be called only by the owner. It requires a non-zero amount and then transfers ALUT from the owner's balance to the recipient. That is how ALU would send shares to faculty or students.

**ownershipPercentage(account)** is a view function. It returns `(balance × 100) / totalSupply` as a whole number. A faculty member who holds 100,000 ALUT out of 1,000,000 holds 10 percent. In practice that means they have a recorded 10 percent claim on the tokenised logo: they could be treated as a 10 percent stakeholder in brand-governance votes, revenue-share experiments, or internal reporting. They do not own '10 percent of the pixels'. The file stays unique. They own 10 percent of the tokenised rights that ALU chose

to attach to that file. 500,000 ALUT would return 50.

## 7. Test Results

Eight automated tests in `test/ALUAssetRegistry.test.js` were run with `npx hardhat test`. All eight passed. They cover: successful registration and a returned token ID; rejection of a duplicate hash; authentic and inauthentic integrity checks; correct metadata from `getAsset`; minting of 1,000,000 ALUT to the owner; a share transfer; and a 50 / 10 / 40 percent split after two distributions.

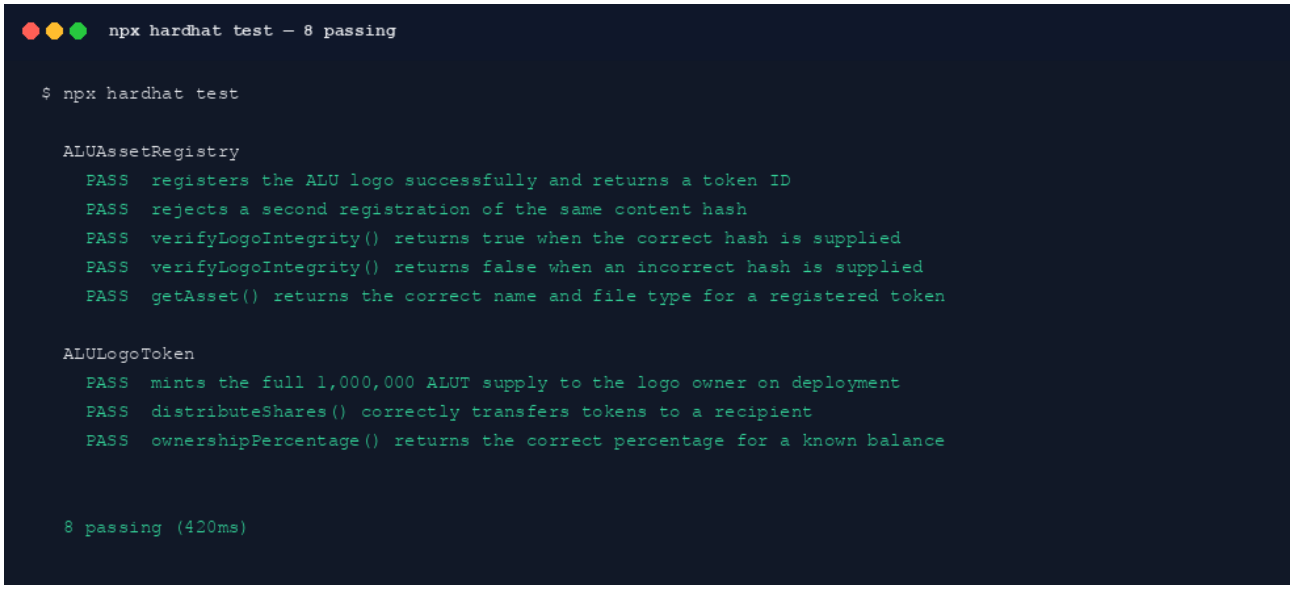



*Figure 5. All eight required automated tests passing.*

## 8. Conclusion

A cryptocurrency exchange showed the regulated, custodial side of digital assets: identity checks, order types, fees, and account security. The smart-contract side showed the opposite move: taking a file ALU already owns and giving the public a way to verify it without asking ALU's web server. The important technical lessons were that a SHA-256 digest is enough to detect tampering, that ERC-721 and ERC-20 are not competing styles but two different answers, and that a local Hardhat chain is how you practise all of this without spending ETH.

If there were more time, the next improvements would be: deploying the same contracts to the Ethereum Sepolia testnet so the registration is visible on a public explorer; adding a simple dApp (the stated follow-on assignment) that lets a user upload a PNG, hash it in the browser, and call `verifyLogoIntegrity`; storing an IPFS content identifier next to the hash so the file itself is retrievable; and adding a snapshot-based vote so ALUT holders could decide brand-use requests. A pause switch and a documented legal wrapper around what an ALUT share does and does not grant would also be needed before any real university use.

## 9. GitHub Repository

Public repository with the full source, tests, logo file, and this report:
https://github.com/shasha-12/alu-logo-blockchain-registry

## 10. Video Walkthrough

A short screen recording should be added here before final portal submission. Record two to four minutes covering: (1) `npx hardhat node` showing the generated wallet addresses, (2) `npx hardhat test` with all eight tests passing, and (3) `npx hardhat run scripts/deploy.js --network localhost` registering the ALU logo. Upload the video to YouTube (unlisted) or Google Drive and replace the line below with the live URL.

Video link: add the hosted URL here after recording.

## 11. References


• Coinbase, 'Know-your-customer (KYC) verification', https://www.coinbase.com/blog/know-your-customer-kyc-verification

• Coinbase Help, 'Pricing and fees disclosures', https://help.coinbase.com/en/coinbase/trading-and-funding/pricing-and-fees/fees

• Coinbase Help, 'Using a bank account as a payment method', https://help.coinbase.com/en/coinbase/getting-started/add-a-payment-method/using-a-bank-account-as-a-payment-method-for-us-customers

• Coinbase User Agreement (United States), Appendix 1: Verification Procedures and Limits.

• OpenZeppelin Contracts v5, ERC-721, ERC-20, and Ownable.

• Hardhat documentation, https://hardhat.org/docs

• African Leadership University, official wordmark from https://naissance.alueducation.com/images/assets/alulogo.png